

Universidad Tecnológica Nacional

Base de Datos II

Consultas de Selección

JOINS y UNION

Autores: Angel Simón, Laura Vélez

Índice

1. Introducción	2
2. El problema del producto cartesiano	3
3. INNER JOIN	5
3.1. ¿Qué significa coincidencia exacta?	5
3.2. Ejemplo práctico: Libros y Categorías	7
4. Anidamiento de JOINS	7
4.1. Ejemplo: Libros con sus Autores	8
4.2. Ejemplo complejo: Lecturas de Usuarios	9
5. LEFT JOIN	11
5.1. ¿Cómo funciona LEFT JOIN?	11
5.2. Ejemplo 1: Autores con y sin Libros	12
5.3. Ejemplo 2: Usuarios con y sin Lecturas	15
5.4. Ejemplo 3: Usuarios sin Lecturas	16
6. RIGHT JOIN	17
6.1. Ejemplo: Medios de Lectura Utilizados y No Utilizados	18
7. FULL JOIN	20
7.1. Ejemplo: Todos los Usuarios y Todos los Medios	20
8. CROSS JOIN	23
9. SELF JOIN	24
9.1. Ejemplo 1: Usuarios que Refirieron a Otros Usuarios	25
9.2. Ejemplo 2: Libros Más Largos que Otros	27
10. UNION	28
10.1. Ejemplo: Estudiantes y Docentes	28

1. Introducción

Hasta el momento, en nuestras prácticas, solamente obtuvimos datos provenientes de una sola tabla de la base de datos. Por ejemplo, si queremos obtener todos los datos de la tabla *Categorias* simplemente es necesario escribir esta consulta y veremos el resultado:

```
SELECT * FROM Categorias
```

IDCategoria	Categoria
1	Novela
2	Terror
3	Fantasía
4	Historia
5	Ciencia Ficción

Si, a partir de estos datos, tuviéramos que elaborar un reporte de libros que incluya sus títulos, años de publicación, cantidad de páginas y categorías, la aparición de un registro con valor de categoría igual a 1 (ID de categoría “Novela”) no le va a representar nada a la persona que lo lea, y hasta es probable que aporte confusión. Pero, si en cambio, mostramos la descripción de la categoría, el reporte será mucho más legible y provechoso. Dejemos por un momento este ejemplo y tratemos de obtener un listado con apellidos, nombres y país de origen de cada uno de los autores (con país de origen nos referimos al nombre del país y no al ID que posee en nuestra tabla *Países*). Como podemos visualizar en el DER y en la siguiente consulta, los apellidos y nombres son datos que podemos extraer directamente de la tabla *Autores*:

```
SELECT * FROM Autores
```

IDAutor	IDPaís	Apellidos	Nombres	Pseudonimo	AñoNacimiento
1	JPN	Murakami	Haruki	NULL	1949
2	GBR	NULL	NULL	Bitmap Books	NULL
3	NULL	King	Stephen	Richard Bachman	1947
4	USA	Crichton	Michael	NULL	1942
"5	JPN	Kawamura	Genki	NULL	1979

Sin embargo, lo más habitual es necesitar datos de dos o más tablas para elaborar listados más descriptivos.

Por ejemplo, es usual querer obtener la descripción principal de un campo a través de su ID. Si estás trabajando con una tabla de categorías y necesitas elaborar un reporte de libros que incluya títulos, años de publicación, cantidad de páginas y categorías, la aparición de un registro con valor de categoría igual a 1 (ID de categoría “Novela”) no le va a representar nada a la persona que lo lea, y hasta es probable que aporte confusión. En cambio, si mostrás la descripción de la categoría, el reporte será mucho más legible y provechoso.

Nota importante

En este apunte trabajaremos con la base de datos Librería. Revisá el Diagrama Entidad-Relación (DER) para entender las relaciones entre las tablas antes de continuar.

2. El problema del producto cartesiano

Supongamos que querés obtener un listado con apellidos, nombres y país de origen de cada uno de los autores. El apellido y el nombre lo obtenés directamente de la tabla Autores, pero para obtener el nombre del país necesitás incorporar la tabla Paises, ya que Autores contiene el ID del país pero no su nombre.

Si simplemente incluís la tabla Paises en la consulta de esta manera:

```
SELECT Autores.Apellidos, Autores.Nombres, Paises.Pais
FROM Autores, Paises;
```

Lo que obtenés es el **producto cartesiano** entre los registros de ambas tablas: cada registro de autores aparece combinado con *cada uno* de los registros de países. Si tenés 5 autores y 3 países, el resultado tendrá 15 filas (5×3), lo cual claramente es incorrecto respecto de lo que querías obtener.

Apellidos	Nombres	País
Murakami	Haruki	Reino Unido
<i>NULL</i>	<i>NULL</i>	Reino Unido
King	Stephen	Reino Unido
Crichton	Michael	Reino Unido
Kawamura	Genki	Reino Unido
Murakami	Haruki	Japón
<i>NULL</i>	<i>NULL</i>	Japón
King	Stephen	Japón
Crichton	Michael	Japón
Kawamura	Genki	Japón
Murakami	Haruki	Estados Unidos
<i>NULL</i>	<i>NULL</i>	Estados Unidos
King	Stephen	Estados Unidos
Crichton	Michael	Estados Unidos
Kawamura	Genki	Estados Unidos

El problema ocurre porque usaste más de una tabla dentro de la cláusula FROM sin especificar que querés todos los registros de autores pero solamente un registro de países: el que se relaciona con cada registro de autor en particular.

Para lograr el resultado deseado es necesario incluir una condición dentro del WHERE basada en la relación existente entre la clave foránea de Autores y la clave primaria de Países:

```
SELECT A.Apellidos, A.Nombres, P.Pais
FROM Autores AS A, Países AS P
WHERE A.IDPais = P.IDPais;
```

Apellidos	Nombres	País
Murakami	Haruki	Japón
<i>NULL</i>	<i>NULL</i>	Reino Unido
Crichton	Michael	Estados Unidos
Kawamura	Genki	Japón

⋮

Aquí podés notar un par de cuestiones. La primera es el uso de **alias** para las tablas: la tabla

Autores se representa con el alias A mientras que Países se representa con P.

Lo segundo es que, a diferencia de la consulta anterior, ahora obtenés los registros tal como los necesitás. Esto se debe a que a cada fila se le aplica la condición del `WHERE`: esta condición indica que el campo `IDPaís` de la tabla Autores debe tener el mismo valor que en el campo `IDPaís` de la tabla Países.

Uso de alias

Los alias para tablas son muy útiles cuando trabajás con múltiples tablas, especialmente cuando tienen nombres largos. Simplifican la escritura y hacen que la consulta sea más legible.

3. INNER JOIN

En el ejemplo anterior logramos obtener los resultados esperados utilizando un `WHERE` obligatoriamente. Si bien el resultado es correcto, por cuestiones de **eficiencia** se recomienda utilizar la cláusula `INNER JOIN` en su lugar.

La función específica de `INNER JOIN` es relacionar dos tablas mediante un campo que sirve como nexo, buscando la **coincidencia exacta** entre los valores de este campo en ambas tablas.

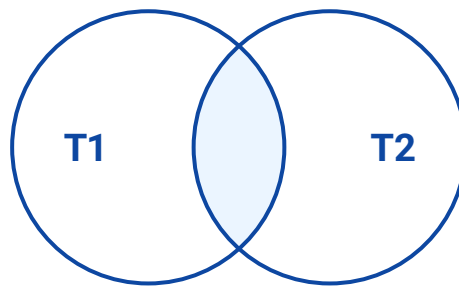
3.1. ¿Qué significa coincidencia exacta?

Significa que `INNER JOIN` devuelve únicamente aquellos registros donde coincida (en las dos tablas) el valor del campo que las une. Es decir, si en una de las tablas el valor no se encuentra, el registro no será devuelto.

La sintaxis de `INNER JOIN` es la siguiente:

```
SELECT T1.Columna1, T2.Columna2
FROM Tabla1 AS T1
INNER JOIN Tabla2 AS T2 ON T1.ID = T2.ID;
```

Si lo pensamos “gráficamente”, como en la teoría de conjuntos, INNER JOIN representa la **intersección** entre las tablas.



Escribamos la consulta anterior reemplazando la cláusula WHERE por INNER JOIN:

```
SELECT A.Apellidos, A.Nombres, P.Pais
FROM Autores AS A
INNER JOIN Países AS P ON A.IDPais = P.IDPais;
```

A diferencia de la consulta anterior donde incluimos más de una tabla en la cláusula FROM, aquí solamente tenemos una (Autores). Luego indicamos que la tabla Autores tendrá una relación con la tabla Países mediante la cláusula INNER JOIN, y finalmente especificamos su nexo (ON): el campo IDPais de Autores y el campo IDPais de Países.

El resultado de esta consulta es:

Apellidos	Nombres	País
Murakami	Haruki	Japón
NULL	NULL	Reino Unido
Crichton	Michael	Estados Unidos
Kawamura	Genki	Japón

⋮

Importante sobre INNER JOIN

Gracias al funcionamiento de INNER JOIN, los registros que tengan NULL en el campo de relación no aparecerán en el resultado. Esta cláusula exige una coincidencia exacta en ambas tablas: el valor de las columnas que se relacionan debe existir y ser el mismo en las dos tablas.

Si tuvieras un valor en IDPais de la tabla Autores que no existe en la tabla Países, o bien tuviese NULL, el registro no será incluido en la selección.

3.2. Ejemplo práctico: Libros y Categorías

Retomemos el ejemplo del inicio. Necesitamos un listado de libros y categorías. Ya conociendo la utilidad del INNER JOIN, escribimos la consulta:

```
SELECT L.Titulo, L.AñoPublicacion, L.Paginas, C.Categoria
FROM Libros AS L
INNER JOIN Categorías AS C ON L.IDCategoria = C.IDCategoria;
```

Título	Año Publicación	Páginas	Categoría
Molly	2023	432	Terror
El visitante	2018	592	Terror
Fin de guardia	2016	448	Terror
Quien pierde paga	2015	434	Terror
Mr Mercedes	2014	437	Terror
La sangre manda	2020	448	Terror
Micro	2011	416	Ciencia Ficción
El Instituto	2019	561	Terror
Salem's Lot	1975	653	Terror
Apocalipsis	1978	1152	Terror
Jurassic Park	1990	448	Ciencia Ficción
If Cats Disappeared from the World	2012	208	Novela
El lector	1995	224	Novela
Despues	2021	272	Terror
Cementerio de animales	1983	416	Terror
El piloto del Danubio	1908	320	Novela
Drácula	1897	418	Terror
Viaje al centro de la Tierra	1864	312	Novela
Las Nieves del Kilimanjaro y otros cuentos	1936	240	Novela
Needful things	1991	736	Terror

4. Anidamiento de JOINS

Es probable que hayas normalizado lo suficiente una base de datos como para que, a la hora de obtener datos, sea necesario recorrer diferentes niveles de relaciones entre más de dos tablas. Esto es posible y lo veremos muy frecuentemente, pero para ello debemos utilizar los JOINS de manera correcta.

4.1. Ejemplo: Libros con sus Autores

Queremos obtener un listado con los siguientes datos de libros: título, cantidad de páginas y, además, nombres, apellidos y seudónimos de sus autores.

Si analizás el DER observarás que los datos de título y cantidad de páginas los obtenés de la tabla Libros, mientras que apellidos, nombres y seudónimos salen de la tabla Autores. Sin embargo, para que el resultado sea correcto, es preciso que los datos de libros y autores estén vinculados entre sí.

Las tablas Libros y Autores tienen una relación de **muchos a muchos**: un autor puede haber escrito más de un libro (o ninguno), y un libro puede haber sido escrito por más de un autor (o no registrar autor). Por este motivo es que la tabla Libros no tiene, por ejemplo, un campo IDAutor, ni la tabla Autores tiene un campo IDLibro.

Para relacionar ambas entidades entra en juego la tabla de unión AutoresLibros. Para este listado necesitamos datos de tres tablas distintas, y para relacionarlas serán necesarios **dos INNER JOINS**:

```
SELECT L.Titulo, L.Paginas, A.Nombres, A.Apellidos, A.Pseudonimo
FROM Libros L
INNER JOIN AutoresLibros AL ON L.IDLibro = AL.IDLibro
INNER JOIN Autores A ON AL.IDAutor = A.IDAutor;
```

El resultado de esta consulta es el siguiente (se muestran sólo algunos registros...)

Título	Paginas	Nombres	Apellidos	Pseudonimo
Corazones en la Atlántida	672	Stephen	King	Richard Bachman
Micro	416	Michael	Crichton	NULL
Jurassic Park	448	Michael	Crichton	NULL
Prey	502	Michael	Crichton	NULL
If Cats Disappeared from the World	208	Genki	Kawamura	NULL
Buscando a Papá Noel	128	Genki	Kawamura	NULL
El lector	224	Bernhard	Schlink	NULL
Las Nieves del Kilimanjaro y otros cuentos	240	Ernest	Hemingway	NULL
Whales: Giants Of The Seas And Oceans	304	Paul	Greenberg	NULL
Armada	384	Ernest	Cline	NULL
Ready Player One	384	Ernest	Cline	NULL
1984	328	Ernest	Cline	NULL
La Granja	144	Ernest	Cline	NULL
El piloto del Danubio	320	Julio	Verne	NULL
Viaje al centro de la Tierra	312	Julio	Verne	NULL

Título	Paginas	Nombres	Apellidos	Pseudonimo
El camino a Francia	308	Julio	Verne	NULL
20.000 Leguas de viaje submarino	416	Julio	Verne	NULL
El viejo y el mar	127	Julio	Verne	NULL
Drácula	418	Bram	Stoker	NULL
El diablo en la botella	112	Robert Louis	Stevenson	NULL
Un escándalo en Bohemia	40	Robert Louis	Stevenson	NULL

⋮

4.2. Ejemplo complejo: Lecturas de Usuarios

Necesitamos un listado que contenga: nombre del usuario, nombre de su país de origen, título del libro que leyó, fecha en que realizó la lectura, nombre del medio de lectura que utilizó y la cantidad de minutos que duró su sesión de lectura. Además, el listado debe estar ordenado por nombre de usuario, título del libro y fecha de lectura, todo de forma ascendente.

Estrategia para consultas complejas

Antes de escribir una consulta compleja, es recomendable dividir la tarea en partes más pequeñas:

1. Analizar el DER y evaluar qué tablas necesitas para extraer los datos
2. Identificar las relaciones existentes entre esas tablas
3. Determinar cuál es el “camino” para llegar a una tabla a partir de otra
4. Identificar los campos que tienen en común para poder unirlos
5. Escribir la cláusula WHERE si existen condiciones de filtrado
6. Quedarte solo con las columnas que te interesan
7. Realizar el ordenamiento según el criterio especificado

Analicemos el requerimiento paso a paso:

Paso 1: Identificar las tablas necesarias

El nombre de usuario se obtiene de la tabla Usuarios (campo NombreUsuario). El nombre del país de origen del usuario se encuentra en la tabla Paises (campo Pais). Para obtener el país a través del usuario usamos la columna IDPais que relaciona ambas tablas.

El título del libro se obtiene de la tabla Libros (Titulo). Pero, ¿es correcto relacionar Libros con Usuarios? No pueden relacionarse directamente ya que la relación entre libros y usuarios es de muchos a muchos. Para ello contamos con la tabla Lecturas, que será el nexo entre los lectores (usuarios) y los libros.

Tanto la fecha de lectura como la duración en minutos las obtenemos de Lecturas, mediante las columnas FechaHora y TiempoEnMinutos respectivamente. Finalmente necesitamos conocer el nombre del medio de lectura: en la tabla Lecturas contamos con el ID del medio de lectura (IDMedio), pero no con su descripción. Por lo tanto, a partir de dicho campo debemos establecer una relación más con la tabla Medios para obtener el campo Nombre.

Paso 2: Escribir el esqueleto de la consulta

```
SELECT * FROM Usuarios U
INNER JOIN Países P ON U.IDPaís = P.IDPaís
INNER JOIN Lecturas LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros LI ON LE.IDLibro = LI.IDLibro
INNER JOIN Medios M ON LE.IDMedio = M.IDMedio
```

Paso 3: Seleccionar solo las columnas necesarias

```
SELECT
U.NombreUsuario, P.País, LI.Título, LE.FechaHora AS 'FechaLectura',
M.Nombre AS 'MedioLectura', LE.TiempoEnMinutos
FROM Usuarios U
INNER JOIN Países P ON U.IDPaís = P.IDPaís
INNER JOIN Lecturas LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros LI ON LE.IDLibro = LI.IDLibro
INNER JOIN Medios M ON LE.IDMedio = M.IDMedio
```

Paso 4: Agregar el ordenamiento

```
SELECT
U.NombreUsuario, P.País, LI.Título, LE.FechaHora AS 'FechaLectura',
M.Nombre AS 'MedioLectura', LE.TiempoEnMinutos
FROM Usuarios U
INNER JOIN Países P ON U.IDPaís = P.IDPaís
INNER JOIN Lecturas LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros LI ON LE.IDLibro = LI.IDLibro
INNER JOIN Medios M ON LE.IDMedio = M.IDMedio
ORDER BY U.NombreUsuario ASC, LI.Título ASC, LE.FechaHora ASC;
```

NombreUsuario	Pais	Título	FechaLectura	MedioLectura	TiempoEnMinutos
Ana	Argentina	20.000 Leguas de viaje submarino	2017-07-17	Ebook	94
Ana	Argentina	20.000 Leguas de viaje submarino	2017-07-18	Tablet	277
Ana	Argentina	20.000 Leguas de viaje submarino	2017-07-20	PC	33
Ana	Argentina	20.000 Leguas de viaje submarino	2017-07-22	Tablet con E-Ink	163
Ana	Argentina	20.000 Leguas de viaje submarino	2017-07-23	Audiolibro	28

⋮

5. LEFT JOIN

Retomemos el ejemplo de autores y países:

```
SELECT A.Apellidos, A.Nombres, P.Pais
FROM Autores AS A
INNER JOIN Paises AS P ON A.IDPais = P.IDPais;
```

Quedó pendiente la pregunta: ¿es posible obtener el registro de Autores cuyo valor en la columna IDPais es NULL?

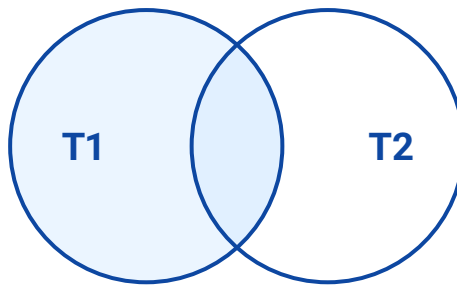
Sí, es posible y para ello utilizamos la cláusula LEFT JOIN, que tiene la siguiente estructura:

```
SELECT T1.Columna1, T2.Columna2
FROM Tabla1 AS T1
LEFT JOIN Tabla2 AS T2 ON T1.ID = T2.ID;
```

5.1. ¿Cómo funciona LEFT JOIN?

Al igual que INNER JOIN, la cláusula LEFT JOIN se utiliza para indicar la relación existente entre dos tablas mediante las columnas que sirven como nexos. Pero, a diferencia de INNER, LEFT JOIN **no exige** que exista el valor de la columna de la tabla de la izquierda en la tabla de la derecha para poder obtener los datos.

Dicho de otro modo, LEFT JOIN devuelve todos los registros de la tabla izquierda (T1), incluyendo las coincidencias con la tabla derecha (T2) pero también los registros que no son coincidentes en T2, ya sea por contener NULL en T1.ID o por no existir un ID en T2 que sea igual a T1.ID. Los campos de T2 que no pudieron ser relacionados con T1 se completan con NULL.



Veamos qué resultados arroja nuestra consulta modificando el `INNER JOIN` por un `LEFT JOIN`:

```
SELECT A.Apellidos, A.Nombres, P.Pais
FROM Autores AS A
LEFT JOIN Países AS P ON A.IDPais = P.IDPais;
```

Apellidos	Nombres	Pais
Murakami	Haruki	Japón
<i>NULL</i>	<i>NULL</i>	Reino Unido
King	Stephen	<i>NULL</i>
Crichton	Michael	Estados Unidos
Kawamura	Genki	Japón

Ahora sí logramos recuperar todos los registros de la tabla de autores, incluyendo aquellos que tienen `NULL` en el campo `IDPais`.

5.2. Ejemplo 1: Autores con y sin Libros

Queremos obtener apellidos y nombres de los autores junto a los títulos y años de publicación de sus libros. Si hay autores sin libros asociados, se deberán listar también.

Realizando un análisis inicial, necesitamos apellidos y nombres de los autores (de la tabla `Autores`) y títulos y años de publicación de los libros (de la tabla `Libros`). Como autores y libros tienen una relación de varios a varios, sabemos que ambas tablas no se relacionan directamente sino que debemos pasar a través de la tabla de unión `AutoresLibros`.

Escribamos la consulta con lo que sabemos hasta aquí:

```
SELECT * FROM Autores AS A
(???) JOIN AutoresLibros AS AL ON A.IDAutor = AL.IDAutor
(???) JOIN Libros AS L ON AL.IDLibro = L.IDLibro;
```

Como se nos indica que los autores sin libros asociados también deben ser listados, significa que hay que traer todos los registros de Autores sin importar si tienen o no correspondencia con los registros de Libros. Eso nos da una idea de que tenemos que utilizar `LEFT JOIN`, y de que Autores será la tabla izquierda.

Ahora bien, ¿cuál de los dos JOINS será `LEFT`? ¿Con AutoresLibros (y Libros queda como `INNER`), con Libros (y AutoresLibros queda como `INNER`), o con ambas?

Una forma de averiguarlo es conocer previamente cuáles son los autores que no tienen libros. Por ejemplo, supongamos que los autores con IDAutor 1, 22 y 25 no tienen libros (no hay registros en AutoresLibros donde aparezca su ID).

IAutor	IDPais	Apellidos	Nombres
1	JPN	Murakami	Haruki
22	FRA	Saint-Exupéry	Antoine de
25	FRA	Yves	Cohat

Nuestra consulta debe devolver, entre otros, los registros con IDAutor 1, 22 y 25, con valores nulos en las tablas de la derecha.

Probemos diferentes combinaciones:

Opción 1: `LEFT` con AutoresLibros, `INNER` con Libros

```
SELECT A.IDAutor, A.Apellidos, A.Nombres, AL.IDAutor, AL.IDLibro,
L.IDLibro, L.Titulo
FROM Autores AS A
LEFT JOIN AutoresLibros AS AL ON A.IDAutor = AL.IDAutor
INNER JOIN Libros AS L ON AL.IDLibro = L.IDLibro
WHERE A.IDAutor IN (1, 2, 3, 22, 25)
ORDER BY A.IDAutor;
```

Esta consulta, si bien trae algunos resultados, no incluye a los autores 1, 22 y 25 que vimos que no tenían libros. Esto sucede porque el `INNER JOIN` con Libros exige que obligatoriamente exista un registro que contenga la vinculación entre IDLibro de Lecturas y IDLibro de Libros.

IDAutor	Apellidos	Nombres	IDAutor	IDLibro	IDLibro	Título
2	NULL	NULL	2	40	40	La historia de Nintendo
2	NULL	NULL	2	41	41	Visual Studio Code Succintly
3	King	Stephen	3	1	1	Holly
3	King	Stephen	3	2	2	El visitante

IDAutor	Apellidos	Nombres	IDAutor	IDLibro	IDLibro	Título
3	King	Stephen	3	3	3	Fin de guardia

Opción 2: INNER con AutoresLibros, LEFT con Libros

Invertir LEFT e INNER tampoco funciona porque seguimos exigiendo coincidencia exacta entre Autores y AutoresLibros.

Opción 3: LEFT con ambas (CORRECTO)

```
SELECT A.IDAutor, A.Apellidos, A.Nombres, AL.IDAutor, AL.IDLibro,
L.IDLibro, L.Título
FROM Autores AS A
LEFT JOIN AutoresLibros AS AL ON A.IDAutor = AL.IDAutor
LEFT JOIN Libros AS L ON AL.IDLibro = L.IDLibro
WHERE A.IDAutor IN (1, 2, 3, 22, 25)
ORDER BY A.IDAutor;
```

IDAutor	Apellidos	Nombres	IDAutor	IDLibro	IDLibro	Título
1	Murakami	Haruki	NULL	NULL	NULL	NULL
2	NULL	NULL	2	40	40	La historia de Nintendo
2	NULL	NULL	2	41	41	Visual Studio Code Succintly
3	King	Stephen	3	1	1	Holly
...						
3	King	Stephen	3	77	77	Corazones en la Atlántida
22	Saint-Exupéry	Antoine de	NULL	NULL	NULL	NULL
25	Yves	Cohat	NULL	NULL	NULL	NULL

Ahora sí, los resultados son los que esperábamos: vemos a todos los autores, incluso los que no escribieron libros, y en esos casos se rellenan con NULL todos los campos de las tablas que se encuentran a la derecha, es decir, AutoresLibros y Libros.

Explicación importante

La explicación de por qué tenemos que usar LEFT JOIN para unir a Autores con AutoresLibros y Libros es:

Autores A INNER JOIN AutoresLibros AL: Esta sentencia busca la coincidencia exacta. Los ID de autores 1, 22 y 25 no existen en AutoresLibros, con lo cual un INNER los excluye de la selección.

(Autores A LEFT JOIN AutoresLibros AL) INNER JOIN Libros L: De la misma manera, aquí se buscan los registros donde coincidan exactamente los valores del campo

IDLibro en las tablas AutoresLibros y Libros. AutoresLibros no contiene, por ejemplo, un registro con IDAutor = 1. Por ende, no puede haber coincidencia exacta entre un registro no existente (NULL) en AutoresLibros y un registro existente en Libros.

Una vez analizado esto y cuando armamos correctamente los JOINS, escribimos la consulta definitiva:

```
SELECT A.Apellidos, A.Nombres, L.Titulo, L.AñoPublicacion
FROM Autores AS A
LEFT JOIN AutoresLibros AS AL ON A.IDAutor = AL.IDAutor
LEFT JOIN Libros AS L ON AL.IDLibro = L.IDLibro;
```

5.3. Ejemplo 2: Usuarios con y sin Lecturas

Necesitamos un listado con nombre de usuario, título del libro leído y fecha de lectura. El listado debe estar ordenado por fecha de lectura y, si hay usuarios que no registraron lecturas, deben visualizarse de todos modos.

Para obtener nombre de usuario debemos acudir a la tabla Usuarios (NombreUsuario), para el título del libro consultamos la tabla Libros (Titulo) y para la fecha de lectura necesitamos la tabla Lecturas (FechaHora).

Los usuarios y los libros tienen una relación de muchos a muchos. Por lo cual, la tabla Lecturas es la tabla de unión entre Usuarios y Libros.

Como los usuarios que no leyeron también deben ser listados, debemos obtener todos los registros de usuarios. Esto incluye a los que hayan leído y a los que no. La consulta finalmente queda así:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora
FROM Usuarios AS U
LEFT JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
LEFT JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
ORDER BY LE.FechaHora ASC;
```

5.4. Ejemplo 3: Usuarios sin Lecturas

Necesitamos obtener el nombre de usuario y nombre de su país de origen de todos aquellos usuarios que no hayan registrado lecturas.

Comencemos evaluando las tablas necesarias: Usuarios (para obtener NombreUsuario), Países (relacionada con Usuarios a través del campo IDPaís, para obtener el campo País), y Lecturas, para recuperar solamente los registros que cumplan con la condición.

Si armamos el “esqueleto” de la consulta tenemos algo como lo que sigue:

```
SELECT U.NombreUsuario, P.Pais
FROM Usuarios AS U
(???) JOIN Países AS P ON U.IDPaís = P.IDPaís
(???) JOIN Lecturas AS L ON U.IDUsuario = L.IDUsuario;
```

En la tabla Usuarios, el campo IDPaís no puede ser nulo y es clave foránea. Por todo esto inferimos que para que exista un registro de usuario, el campo IDPaís debe estar completo con un valor que exista en la tabla Países. Luego, el tipo de unión que vincula a Usuarios con Países es de tipo INNER:

```
SELECT U.NombreUsuario, P.Pais
FROM Usuarios AS U
INNER JOIN Países AS P ON U.IDPaís = P.IDPaís
(???) JOIN Lecturas AS L ON U.IDUsuario = L.IDUsuario;
```

Respecto de la tabla Lecturas, sabemos que no todos los registros de Usuarios deben vincularse con ella, ya que puede haber usuarios que no hayan registrado lecturas:

```
SELECT U.NombreUsuario, P.Pais
FROM Usuarios AS U
INNER JOIN Países AS P ON U.IDPaís = P.IDPaís
LEFT JOIN Lecturas AS L ON U.IDUsuario = L.IDUsuario;
```

Finalmente debemos incorporar a la consulta una cláusula WHERE que contenga la condición para que obtenga solamente los registros de usuarios que no registraron lecturas. Es decir, debemos indicar de alguna manera que, para listar un registro de usuario, su ID no debe aparecer en ningún registro de la tabla Lecturas. ¿Y con qué valor completa LEFT JOIN las columnas de la tabla derecha en caso de no poder establecer la relación? Con NULL:

```
SELECT U.NombreUsuario, P.Pais
FROM Usuarios AS U
INNER JOIN Paises AS P ON U.IDPais = P.IDPais
LEFT JOIN Lecturas AS L ON U.IDUsuario = L.IDUsuario
WHERE L.IDLectura IS NULL;
```

NombreUsuario	Pais
PabloFernandez	España
AlejandroGarcia	España
CarmenLopez	España
LuciaMartinez	España

Entendiendo la condición

¿Cómo se entiende esta relación y su condición? Al establecer una relación de tipo LEFT JOIN entre Usuarios y Lecturas, en principio solicitamos todos los registros de Usuarios, se encuentren o no en Lecturas.

Al pedir que IDLectura en Lecturas sea NULL, como dicho campo es la clave primaria de esa tabla, lo que queremos decir es que no debe existir un registro en Lecturas que satisfaga la restricción de clave foránea con Usuarios. Esto equivale a decir que el IDUsuario de Usuarios no debe estar presente en Lecturas o, en otras palabras, que el usuario no está registrado en Lecturas.

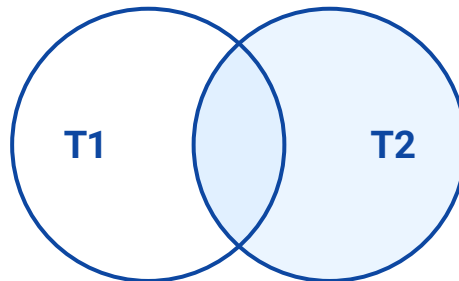
6. RIGHT JOIN

Esta cláusula tiene un funcionamiento similar al de LEFT JOIN, pero podría decirse que “al revés”. Es decir, para obtener los datos no exige que el valor de los campos (que actúan como nexos) de la tabla de la derecha existan en la tabla de la izquierda.

```
SELECT T1.Columna1, T2.Columna2
FROM Tabla1 AS T1
RIGHT JOIN Tabla2 AS T2 ON T1.ID = T2.ID;
```

Lo que hará una consulta como esta es devolver todos los registros de T2. En los casos que haya coincidencia con T1, los registros de T1 serán incluidos. Caso contrario, se completarán los valores de los campos de T1 con NULL.

Si lo graficamos en un Diagrama de Venn se vería de la siguiente manera:



6.1. Ejemplo: Medios de Lectura Utilizados y No Utilizados

Queremos un listado que contenga el nombre de usuario, título del libro leído, fecha de lectura y medio de lectura. Además, incluiremos en el listado a los medios de lectura que no han sido utilizados.

Empezamos relevando las tablas necesarias: Usuarios (campo NombreUsuario), Libros (Titulo), Lecturas (FechaHora), Medios (Nombre). La tabla Medios se relaciona con Lecturas a través del campo IDMedio.

Armamos un bosquejo de la consulta:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
(???) JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
(???) JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
(???) JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

Ahora pensemos cómo se relacionan las tablas:

Usuarios y Lecturas deben tener coincidencia exacta, ya que de lo contrario se indicaría que se incluyan los usuarios que no registren lecturas:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
INNER JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
(???) JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
```

```
(???) JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

Entre Lecturas y Libros también necesitamos una coincidencia exacta, porque tampoco se especifica que incluyamos los registros de libros que no hayan sido leídos:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
INNER JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
(???) JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

En el caso de Medios y Lecturas, sí nos piden que se incluyan los medios que no hayan sido utilizados. Entonces ahora sí tenemos que usar RIGHT JOIN:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
INNER JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
RIGHT JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

Notá que no tenemos necesidad de incluir una condición ya que, a diferencia de ejemplos anteriores, no se pide excluir un conjunto de resultados en particular, sino que obtendremos todos los registros de Medios, sin importar que hayan sido utilizados o no. O sea, esta vez no preguntaremos por el cumplimiento de la restricción de clave foránea que existe entre ambas tablas. Ejecutamos la consulta y obtenemos el siguiente resultado:

NombreUsuario	Titulo	FechaHora	NombreMedio
JavierRuiz	Cementerio de animales	2024-05-30	Braille Ebook
NULL	NULL	NULL	Lector de pantalla
NULL	NULL	NULL	Libro de papel en Braille

⋮

Para corroborar que ciertos medios nunca fueron utilizados, podemos hacer una consulta similar a la que vimos anteriormente con LEFT JOIN: obtendremos los nombres de los medios cuyo ID no aparezca en ningún registro de la tabla Lecturas:

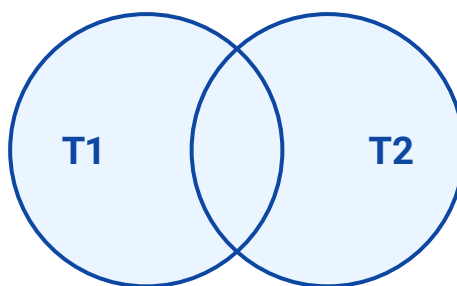
```
SELECT M.Nombre AS NombreMedio
FROM Medios AS M
LEFT JOIN Lecturas AS L ON M.IDMedio = L.IDMedio
WHERE L.IDLectura IS NULL;
```

Este es el resultado:

	NombreMedio
Lector de pantalla	
Libro de papel en Braille	

7. FULL JOIN

Como su nombre lo indica, `FULL JOIN` permite obtener el listado completo de registros de dos tablas relacionadas, conformados por la combinación de `LEFT JOIN` y `RIGHT JOIN`. Esto es, `FULL JOIN` devolverá todos los registros de la tabla izquierda más todos los registros de la tabla derecha, con sus respectivas coincidencias. Los campos que no coinciden serán rellenados con `NULL`.



```
SELECT T1.Columna1, T2.Columna2
FROM Tabla1 AS T1
FULL JOIN Tabla2 AS T2 ON T1.ID = T2.ID;
```

7.1. Ejemplo: Todos los Usuarios y Todos los Medios

Siguiendo en la línea de los casos anteriores, confeccionaremos un listado que contenga nombre de usuario, título de libro leído, fecha de lectura, nombre del medio utilizado, y la condición de que, además de mostrar todos los medios (utilizados o no) también se listen todos los usuarios, hayan leído o no.

Escribiremos la consulta partiendo de la anterior:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
```

```
INNER JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
RIGHT JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

La relación entre Medios y Lecturas en principio queda igual. Nos detendremos un momento en la de Usuarios y Lecturas, ya que para obtener los registros de Usuarios no necesariamente tienen que tener registrada una lectura:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
LEFT JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
INNER JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
RIGHT JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

Con este cambio desligamos la obligatoriedad de los usuarios respecto de las lecturas, pero si dejamos INNER JOIN en la relación Lecturas-Libros, estamos exigiendo que obligatoriamente exista un registro que contenga la vinculación entre IDLibro de Lecturas y IDLibro de Libros. Esto contradice lo anterior: no corresponde pedir la coincidencia exacta entre Lecturas y Libros:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
LEFT JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
LEFT JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
RIGHT JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

Finalmente, dado que necesitamos todos los registros de Usuarios (tabla izquierda) pero también todos los de Medios (tabla derecha), deducimos que el tipo de unión que corresponde a Medios con las tablas que se encuentran a su izquierda es FULL:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
LEFT JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
LEFT JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
FULL JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

NombreUsuario	Titulo	FechaHora	NombreMedio
PabloFernandez	NULL	NULL	NULL
AlejandroGarcia	NULL	NULL	NULL
CarmenLopez	NULL	NULL	NULL
LuciaMartinez	NULL	NULL	NULL
NULL	NULL	NULL	Lector de pantalla
NULL	NULL	NULL	Libro de papel en Braille

Si ahora queremos obtener el mismo listado anterior pero con la condición de que se listen todos los registros de Libros (hayan sido leídos o no), tenemos que modificar levemente la consulta, quedando de esta manera:

```
SELECT U.NombreUsuario, LI.Titulo, LE.FechaHora, M.Nombre AS NombreMedio
FROM Usuarios AS U
LEFT JOIN Lecturas AS LE ON U.IDUsuario = LE.IDUsuario
FULL JOIN Libros AS LI ON LE.IDLibro = LI.IDLibro
FULL JOIN Medios AS M ON LE.IDMedio = M.IDMedio;
```

El resultado de la consulta es el siguiente:

NombreUsuario	Titulo	FechaHora	NombreMedio
PabloFernandez	NULL	NULL	NULL
AlejandroGarcia	NULL	NULL	NULL
CarmenLopez	NULL	NULL	NULL
LuciaMartinez	NULL	NULL	NULL
NULL	Cantar de Mio Cid	NULL	NULL
NULL	NULL	NULL	Lector de pantalla
NULL	NULL	NULL	Libro de papel en Braille

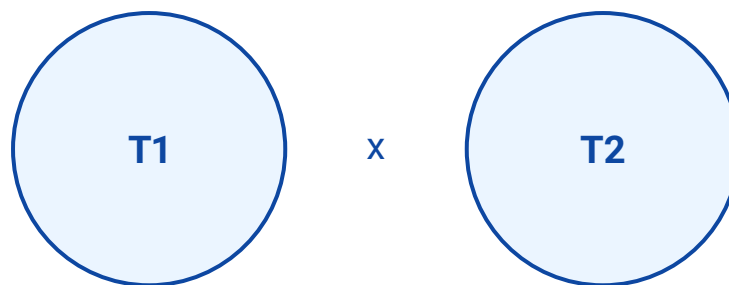
Explicación de FULL JOIN múltiples

Tenemos LEFT JOIN en la relación de Libros y sus tablas de la izquierda (Usuarios y Lecturas) pues queremos recuperar todos los registros de usuarios, aunque no tengan correspondencia. Luego incorporamos un RIGHT JOIN en la relación de Libros y sus tablas de la izquierda porque necesitamos obtener todos los registros de libros, leídos o no. Esto convierte a la combinación de LEFT y RIGHT en FULL.

Lo mismo sucede con Medios, que se relaciona con LEFT JOIN con sus tablas de la izquierda (Usuarios, Lecturas y Libros) porque queremos obtener todos los usuarios, hayan leído o no, y todos los libros, hayan sido leídos o no. Pero a su vez también queremos todos los medios, utilizados o no, con lo cual también hay una unión de tipo RIGHT JOIN. Nuevamente, la combinación de LEFT y RIGHT convierte a la vinculación con Medios en un FULL JOIN.

8. CROSS JOIN

Este tipo de unión no suele ser frecuente, pero es interesante conocerla ya que permite unir tablas sin restricciones, es decir, las tablas pueden estar relacionadas (con una clave foránea) o no. Consiste en el **producto cartesiano** entre dos tablas: si aplicamos `CROSS JOIN` a las tablas T1 y T2, obtendremos todos los registros que se pueden formar combinando cada registro de T1 con cada registro de T2.



La sintaxis de `CROSS JOIN` es:

```
SELECT T1.Columna1, T2.Columna2
FROM Tabla1 AS T1
CROSS JOIN Tabla2 AS T2;
```

Un ejemplo de `CROSS JOIN` lo vimos al principio de este apunte, donde se combinaba cada registro de Autores con cada uno de los registros de Países. Sin embargo, fue un resultado accidental. Ahora sí queremos obtener la combinación de todos los autores y todos los países. La cláusula correcta para obtener este resultado es `CROSS JOIN`:

```
SELECT A.IDAutor, A.Apellidos, A.Nombres, P.Pais
FROM Autores AS A
CROSS JOIN Países AS P;
```

El resultado obtenido:

IDAutor	Apellidos	Nombres	Pais
1	Murakami	Haruki	Argentina
2	NULL	NULL	Argentina
3	King	Stephen	Argentina
4	Crichton	Michael	Argentina
5	Kawamura	Genki	Argentina
⋮			
1	Murakami	Haruki	Alemania

IDAutor	Apellidos	Nombres	País
2	NULL	NULL	Alemania
3	King	Stephen	Alemania
4	Crichton	Michael	Alemania
5	Kawamura	Genki	Alemania

⋮

15	Amicis	Edmundo de	España
16	Druon	Maurice	España
17	Eco	Umberto	España
18	Pérez Rodríguez	Uxío	España
19	Alighieri	Dante	España
20	Bourdain	Anthony	España
21	Balmaceda	Daniel	España
22	Saint-Exupéry	Antoine de	España

⋮

Importante sobre CROSS JOIN

Notá que en este tipo de JOIN no se debe indicar la columna que actúa como nexos entre las dos tablas, ya que podría no existir tal columna: CROSS JOIN une a las tablas por defecto, sin exigir que tengan una relación establecida con una clave foránea.

Por caso, también podríamos unir con CROSS a las tablas Categorías y Países, que, según podemos ver en el DER, no tienen ninguna relación entre sí:

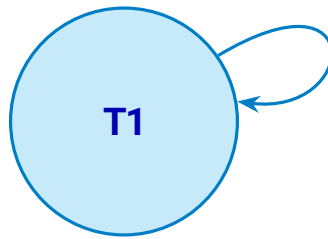
```
SELECT C.*, P.*
FROM Categorías AS C
CROSS JOIN Países AS P;
```

IDCategoría	Categoría	IDPaís	País
1	Novela	ARG	Argentina
2	Terror	ARG	Argentina
3	Fantasía	ARG	Argentina
4	Historia	ARG	Argentina
5	Ciencia Ficción	ARG	Argentina

9. SELF JOIN

Si bien SELF JOIN no es considerado un JOIN propiamente dicho, y no se utiliza explícitamente la palabra SELF para hacer referencia a él, se conoce con este término a un tipo de JOIN en el

que **una tabla se une consigo misma**. Aunque en principio parece carecer de sentido, suele ser muy frecuente la necesidad de recurrir al SELF JOIN.



Se podría representar la sintaxis básica de SELF JOIN de la siguiente manera:

```
SELECT T1.ID, T2.Nombre
FROM Tabla1 AS T1
LEFT JOIN Tabla1 AS T2 ON ...;
```

Como podemos observar, se seleccionan campos de la misma tabla (T1), pero, para distinguir el rol que cumple cada uno dentro de la entidad, se utilizan alias.

9.1. Ejemplo 1: Usuarios que Refirieron a Otros Usuarios

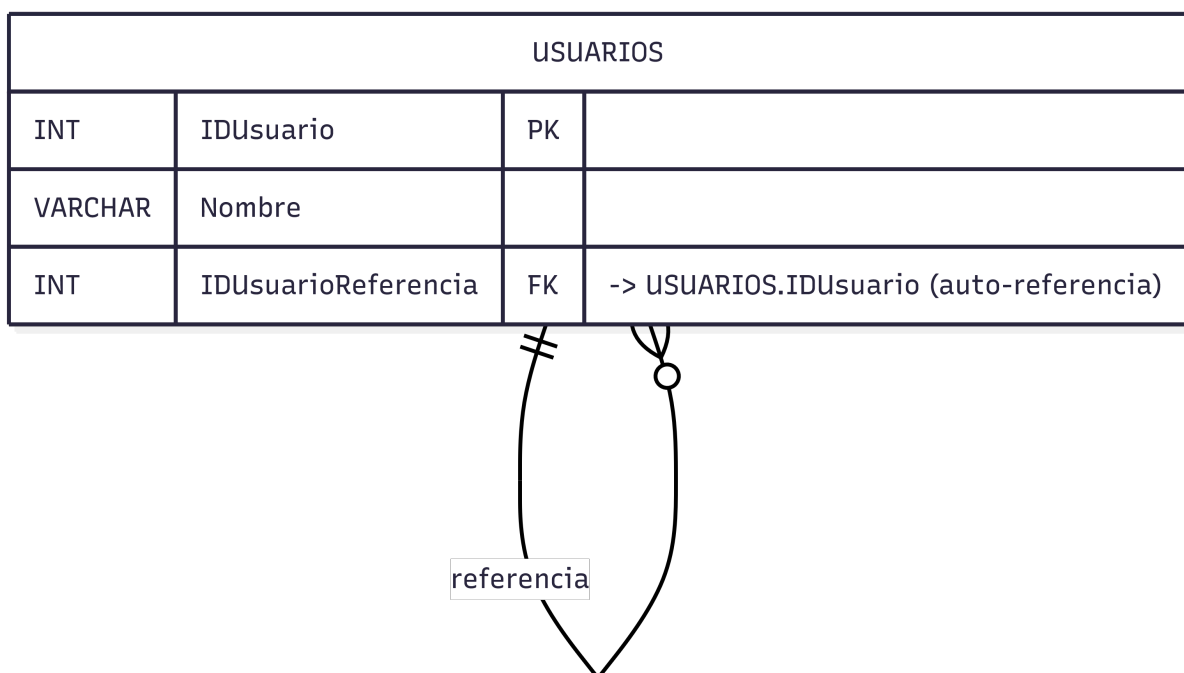


Figura 1. Tabla de Usuarios con una FK a sí misma

Pensemos en una tabla que representa los usuarios de una aplicación web, por ejemplo una red social. La misma solamente puede accederse si alguien te invita. Es decir, necesitás un usuario que te refiera para poder ser un usuario. En estos casos tenemos una relación de Usuarios a Usuarios.

Supongamos que tenemos una tabla Usuarios con los campos IDUsuario, Nombre e IDUsuarioReferencia. El campo IDUsuarioReferencia es una Foreign Key que acepta valores nulos porque claramente al menos un usuario debe poder ser creado sin tener IDUsuarioReferencia.

Veamos posibles datos para la tabla:

IDUsuario	Nombre	IDUsuarioReferencia
1	Administrador	<i>NULL</i>
2	Fernanda	1
3	Mariano	1
4	Abel	2
5	Gladys	4

Podemos observar que al usuario Administrador no lo refirió nadie. Por eso tiene NULL en IDUsuarioReferencia. Luego, a Fernanda y Mariano los refirió el usuario Administrador. De allí que su IDUsuarioReferencia es 1. Abel fue referido por Fernanda, y Gladys por Abel. La idea es poder realizar una consulta que muestre el IDUsuario, el nombre del usuario y el nombre del usuario que lo refirió.

Es decir, realizar una consulta que muestre el IDUsuario, el nombre del usuario y el nombre del usuario que lo refirió. La consulta sería como la siguiente:

```
SELECT U.IDUsuario, U.Nombre, UsuarioReferencia.Nombre AS
'NombreUsuarioReferencia'
FROM Usuarios AS U
LEFT JOIN Usuarios AS UsuarioReferencia
ON U.IDUsuarioReferencia = UsuarioReferencia.IDUsuario;
```

Y el resultado obtenido:

IDUsuario	Nombre	NombreUsuarioReferencia
1	Administrador	<i>NULL</i>
2	Fernanda	Administrador
3	Mariano	Administrador
4	Abel	Fernanda
5	Gladys	Abel

9.2. Ejemplo 2: Libros Más Largos que Otros

Queremos un listado que por cada libro muestre su título y la cantidad de páginas, junto con el título y la cantidad de páginas de todos los libros que tengan más páginas que él. Es decir que por cada libro L queremos el título y las páginas, y queremos el título y las páginas de cada libro Lx que tenga más páginas que L.

Este es otro caso de un JOIN sobre la misma tabla, en este caso la de libros, con la variante de que la condición del JOIN no va a preguntar una igualdad como solemos usar sino que utilizará el operador mayor:

```
SELECT
L.Titulo,
L.Paginas,
LibroMasLargo.Titulo AS 'TituloDeLibroMasLargo',
LibroMasLargo.Paginas AS 'PaginasDeLibroMasLargo'
FROM Libros L
INNER JOIN Libros AS LibroMasLargo ON LibroMasLargo.Paginas > L.Paginas
ORDER BY L.Paginas DESC, LibroMasLargo.Paginas DESC;
```

Podemos inferir que el libro más largo aparecerá primero. Luego le seguirá el segundo libro más largo, y este tendrá un registro que muestre el libro que lo supera. Los libros con varios libros más largos aparecerán múltiples veces, una por cada libro que los supere. Para mejorar la lectura, en la consulta llamamos LibroMasLargo a la tabla en lugar de Lx como se hizo en el ejemplo.

Titulo	Paginas	TituloDeLibroMasLargo	PaginasDeLibroMasLargo
It	1138	Apocalipsis	1152
La cúpula	1074	Apocalipsis	1152
La cúpula	1074	It	1138
Needful things	736	Apocalipsis	1152
Needful things	736	It	1138
Needful things	736	La cúpula	1074
History of food	728	Apocalipsis	1152
History of food	728	It	1138
History of food	728	La cúpula	1074
History of food	728	Needful things	736
Corazones en la Atlántida	672	Apocalipsis	1152
Corazones en la Atlántida	672	It	1138
Corazones en la Atlántida	672	La cúpula	1074

⋮

Non Equi Join

Los JOINS que no utilizan el operador de igualdad se denominan **Non Equi Join**.

10. UNION

El operador `UNION` en SQL se utiliza para combinar los resultados de dos o más consultas `SELECT` en un solo conjunto de resultados. Es especialmente útil cuando deseamos juntar información de diferentes tablas o consultas que tienen la misma cantidad y tipo de columnas en su salida.

Al ejecutar un `UNION`, las filas duplicadas se eliminan automáticamente, mostrando solamente valores únicos. Si queremos incluir todos los registros, incluso los duplicados, se utiliza `UNION ALL`.

10.1. Ejemplo: Estudiantes y Docentes

Vamos a suponer que en una base de datos con la información de un instituto disponemos de las tablas de `Estudiantes` y `Docentes`. Por supuesto, cada tabla contiene información distinta que representa a cada entidad.

La institución quiere hacer una reunión e invitar a todos los estudiantes que se hayan inscripto en el año 2025 y además a todos los docentes. Para ello necesita un listado con Apellidos, Nombres y Mails de todos los docentes y estudiantes que correspondan para hacer las invitaciones a la reunión.

El problema es que queremos un solo listado y tenemos dos tablas que no tienen relación entre sí. No hay join posible aquí. Lo que necesitamos es la unión de las tablas. Es decir, unir horizontalmente dos consultas de selección distintas.

Hipotéticamente tendremos 5 estudiantes y 3 docentes. Para facilitar la visualización del ejemplo:

Estudiantes

Legajo	Apellidos	Nombres	Email	FechaInscripcion
1	González	María	maria.gonzalez@email.com	2023-02-15
2	Pérez	Carlos	carlos.perez@email.com	2025-03-10
3	Rodríguez	Ana	ana.rodriguez@email.com	2025-01-20
4	López	Jorge	lopez.jorge@mail.com	2025-04-01
5	Martínez	Lucía	lucia.martinez@email.com	2023-02-28

Docentes

ID	Apellidos	Nombres	Email	Sueldo	IDCategoría
1	Fernández	Laura	laura.fernandez@email.com	50000.50	1
2	García	Santiago	santiago.garcia@email.com	45000.00	2
3	Romero	Marta	romero.marta@email.com	40000.00	3

Si ejecutamos la consulta siguiente que realiza la unión de ambas tablas, obtendremos el listado que necesitamos:

```
SELECT Apellidos, Nombres, Email FROM Estudiantes
WHERE YEAR(FechaInscripcion) = 2025
UNION
SELECT Apellidos, Nombres, Email FROM Docentes;
```

El resultado es el esperado. En una sola consulta hemos unido la información de dos consultas de selección.

Apellidos	Nombres	Email
Fernández	Laura	laura.fernandez@email.com
García	Santiago	santiago.garcia@email.com
López	Jorge	lopez.jorge@mail.com
Pérez	Carlos	carlos.perez@email.com
Rodríguez	Ana	ana.rodriguez@email.com
Romero	Marta	romero.marta@email.com

Si quisiéramos indicar el rol de cada persona “Estudiante” o “Docente” es tan simple como agregar literalmente esa información en sus respectivos selects:

```
SELECT Apellidos, Nombres, Email, 'Estudiante' AS Rol FROM Estudiantes
WHERE YEAR(FechaInscripcion) = 2025
UNION
SELECT Apellidos, Nombres, Email, 'Docente' AS Rol FROM Docentes;
```

Ahora obtendremos el siguiente listado:

Apellidos	Nombres	Email	Rol
Fernández	Laura	laura.fernandez@email.com	Docente
García	Santiago	santiago.garcia@email.com	Docente
López	Jorge	lopez.jorge@mail.com	Estudiante
Pérez	Carlos	carlos.perez@email.com	Estudiante
Rodríguez	Ana	ana.rodriguez@email.com	Estudiante
Romero	Marta	romero.marta@email.com	Docente

Reglas importantes de UNION

Para que UNION funcione correctamente:

- Las consultas deben tener la misma cantidad de columnas
- Las columnas deben ser del mismo tipo de dato (o compatibles)
- El orden de las columnas debe coincidir
- Los nombres de las columnas se toman de la primera consulta